

4.4 多指令流出技术

前面介绍的技术能够减少或消除数据冲突和控制冲突导致的停顿,使 CPI 尽可能达到为 1 的理想情况。虽然希望 CPI 越小越好,但如果是每次只流出一条指令(单流出),CPI 就不可能小于 1。如果想进一步提高性能,使 CPI 小于 1,就必须采用多流出技术(multiple issue),在每个时钟周期流出多条指令。单流出和多流出处理机执行指令的时空图对比如图 4.11 所示。

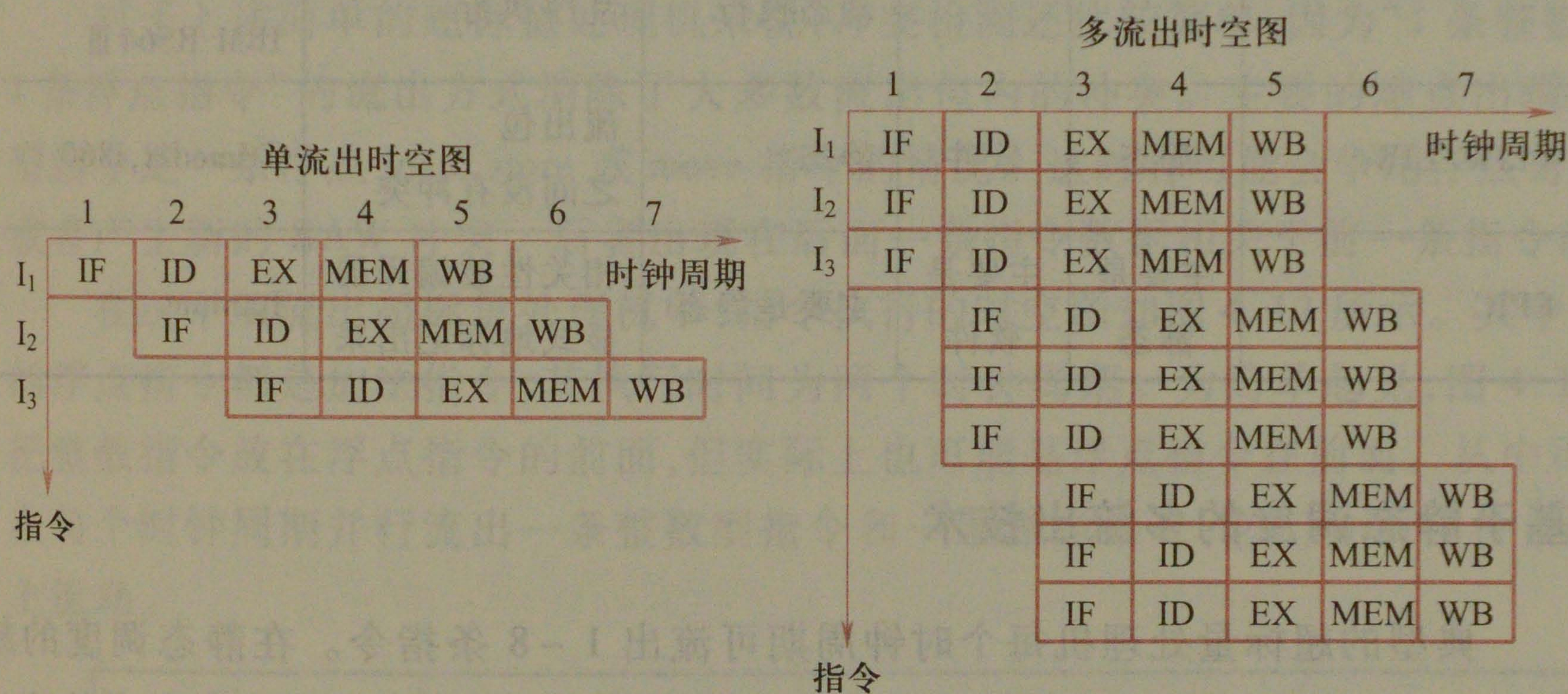


图 4.11 单流出和多流出处理机执行指令的时空图

多流出处理机有两种基本风格:超标量(superscalar)和超长指令字 VLIW (Very Long Instruction Word)。超标量在每个时钟周期流出的指令条数不固定,依代码的具体情况而定,不过有个上限。设这个上限为 n ,就称该处理机为 n -流出。对于超标量处理机,既可以通过编译器进行静态调度,也可以基于 Tomasulo 算法进行动态调度。静态调度的超标量处理机一般采用按序执行,而动态调度的处理机一般采用乱序执行。

与超标量处理机不同,VLIW 处理机在每个时钟周期流出的指令条数是固定的,这些指令构成一条长指令或者一个指令包,在这个指令包中,指令之间的并行性是通过指令显式地表示出来的。这种处理机的指令调度是由编译器静态完成的。

超标量处理机与 VLIW 处理机相比有两个优点:

① 超标量结构对程序员是透明的,因为处理机能自己检测下一条指令能否流出,从而不需要重新排列指令来满足指令的流出。

② 即使是没有经过编译器针对超标量结构进行调度优化的代码或是旧的编译器生成的代码也可以运行,当然运行的效果不会很好。要想达到很好的效果,方法之一就是使用动态超标量调度技术。

表 4.2 列出了一些基本的多流出技术、这些技术的特点以及采用这些技术的处理机例子。

表 4.2 各种多流出技术的特点以及采用这些技术的处理机

技术	流出结构	冲突检测	调度	主要特点	处理机实例
超标量(静态)	动态	硬件	静态	顺序执行	Sun UltraSPARC II / III
超标量(动态)	动态	硬件	动态	部分乱序执行	IBM Power2
超标量(猜测)	动态	硬件	带有猜测的动态执行	带有猜测的乱序执行	Pentium III / 4, MIPS R10K, Alpha 21264, HP PA 8500, IBM RS64 III
VLIW/LIW	静态	软件	静态	流出包之间没有冲突	Trimedia, i860
EPIC	主要是静态	主要是软件	主要是静态	相关性被编译器显式地标记出来	Itanium

4.4.1 基于静态调度的多流出技术

典型的超标量处理机每个时钟周期可流出 1 ~ 8 条指令。在静态调度的超标量处理机中,指令按序流出。所有的冲突检测都在流出时进行,由硬件检测当前流出的指令之间是否存在冲突以及当前流出的指令与正在执行的指令是否有冲突。如果在当前流出的指令序列中,某条指令存在着上述冲突,那么就只流出该指令之前的指令。

例如,一个 4-流出的静态调度超标量处理机,在取指令阶段,流水线将从取指令部件收到 1 ~ 4 条指令,在一个时钟周期内,这些指令有可能是全部都能流出,也可能是只有一部分能流出。不妨把这些指令称为流出包。对于流出包中的任意一条指令来说,如果该指令的流出会与该流出包或者已经在执行中的某条指令产生结构冲突或者数据冲突,那么该指令就不能流出。这些检测是由流出部件完成的。由于这些检测比较复杂,难以在一个时钟周期内完成,所以在许多静态调度的超标量处理机和所有动态调度的超标量处理机中都是将之分成多个流水段,按流水方式工作。例如,可以让第一个段只进行流出包内的冲突检测,选出初步判定可以流出的指令,而在第二段检测所选出的指令与正在执行的指令是否有冲突。

当提高指令流出的速率时,指令流出段有可能会成为指令流水线的瓶颈。虽然把流出段拆分为两段还是比较直观的,但如何进一步分段就不是那么明显了。所以,指令流出段很可能会成为限制超标量处理机的时钟频率提高的一个因素。

如果 MIPS 处理机按超标量方式工作,结果将会怎样? 假设每个时钟周期可以流

出1条整数型指令和1条浮点操作指令,其中把load指令、store指令、分支指令也归类为整数型指令。与任意的双流出相比,把整数指令和浮点指令结合流出是简单了不少,对硬件的要求也没那么高。这种配置和HP 7100处理机中采用的结构很相似,Intel的Pentium也有类似的结构,只不过它是针对整数运算的两路超标量。

为了实现每个时钟周期流出两条指令,显然是要能够同时取两条指令(64位),也要能同时译码两条指令(64位)。对指令的处理包括以下3个步骤:首先从Cache中取两条指令,然后确定哪几条指令可以流出(0条~2条指令),最后把它们发送到相应的功能部件。取两条指令还比较容易实现,若要取更多的指令,所要进行的处理就复杂多了。高性能超标量处理机一般是依靠一个独立的指令预取部件来提供足够的指令流。

对于上述简单的超标量处理机来说,冲突检测还比较简单,因为“1条整数型指令+1条浮点指令”的流出方式消除了大多数流出包内的冲突。主要的难点出现在当整数型指令是一条浮点load、store或move指令的情况。这时有可能会争用浮点寄存器端口或者产生新的RAW冲突。后者出现在后面一条指令数据相关于前一条指令的时候。

在这个双流出超标量处理机中,指令执行的时空图如图4.12所示。其中假设所有的浮点指令都是加法指令,其执行时间为两个时钟周期。为简单起见,图4.12中总是把整数指令放在浮点指令的前面,但实际上也可能是浮点指令在前面。从中可以看出,在每个时钟周期并行流出一条整数型指令和一条浮点指令,它们分别在各自的流水线上流动。

指令类型	流水线工作情况							
整数指令	IF	ID	EX	MEM	WB			
浮点指令	IF	ID	EX	EX	MEM	WB		
整数指令		IF	ID	EX	MEM	WB		
浮点指令		IF	ID	EX	EX	MEM	WB	
整数指令			IF	ID	EX	MEM	WB	
浮点指令			IF	ID	EX	EX	MEM	WB
整数指令				IF	ID	EX	MEM	WB
浮点指令				IF	ID	EX	EX	MEM

图 4.12 双流出超标量流水线中指令的执行过程

这个双流出超标量处理机大大提高了浮点指令流出的速率。与此相匹配,还要提高其浮点运算能力,可以采用流水浮点运算部件,也可以采用多个独立的浮点运算部件。否则,浮点部件会成为整条流水线的瓶颈,丧失双流出带来的好处。

采用“1条整数型指令+1条浮点指令”并行流出的方式,需要增加的硬件很少。这

是因为整数指令和浮点指令使用不同的寄存器组和不同的功能部件。当然,冲突检测是因为整数指令和浮点指令使用不同的寄存器组和不同的功能部件。当然,冲突检测逻辑电路的增加是少不了的。此外,由于把所有 load 和 store 指令归类到整数型指令中,所以会出现浮点 load 或浮点 store 指令与浮点操作指令并行流出的情况,这会导致对浮点寄存器的访问冲突。因此需要增设一个浮点寄存器的读/写端口。还有,由于流水线中的指令多了一倍,定向路径也要增加。

另外还存在一个限制超标量流水线的性能发挥的障碍。在前述简单的流水线中,load 指令有一个时钟周期的延迟,这就使得紧跟其后,并使用其结果的指令要停顿一个时钟周期后才可以流出。在超标量流水线中,load 指令的结果显然不能被同时流出的指令所使用,也不能被下一个周期流出的两条指令所使用(因 load 的延迟),所以 load 后续 3 条指令都不能使用其结果,否则就会引起停顿。再来看看分支延迟。在前述简单的流水线中,分支延迟为一个时钟周期。如果分支指令是流出包中的第一条指令,则其延迟是 3 个时钟周期;否则就是流出包中的第二条指令,其延迟就是两个时钟周期。

为了能有效地利用超标量处理机所具有的并行性,需要采用更有效的编译技术或者硬件调度技术。如果不采用这些技术,超标量技术所能带来的性能上的提高可能很小。

4.4.2 基于动态调度的多流出技术

在多流出处理机中,动态调度技术是提高性能的一种方法。动态调度不仅在有数据冲突的情况下能提高性能,而且还有可能克服指令流出所受的限制。换句话说,尽管从硬件的角度来看,在每个时钟周期最多只能启动一个整数操作和一个浮点操作的执行,但是动态调度可以使得在指令流出时不受这个限制,至少在保留站被全部占用之前是如此。

假如要对 Tomasulo 算法进行扩展,使之能支持双流出超标量流水线,又不想乱序地向保留站流出指令,因为这会破坏程序语义,为了充分利用动态调度的优点,应该去掉每个时钟周期只能流出一条整数指令和一条浮点操作指令的限制。但这会大大地增加指令流出的硬件复杂度。

可以采用另一种比较简单的方法:将整数所用的表结构与浮点所用的表结构分离开,分别进行处理,这样就可以同时流出一条浮点指令和一条整数指令到各自的保留站。

有两种不同的方法可以用来在采用动态调度的处理机中实现多流出。它们都是建立在这样一个认识之上的:关键在于对保留站的分配和对流水线控制表格的修改。一种方法是在半个时钟周期里完成流出步骤,这样一个时钟周期就能处理两条指令。另一种方法是设置一次能同时处理两条指令的逻辑电路。现代的流出 4 条或 4 条以上指令的超标量处理机经常是两种方法都采用。

例 4.4 对于采用了 Tomasulo 算法和多流出技术的 MIPS 流水线,考虑以下简单循

环的执行。该程序把 F2 中的标量加到一个向量的每个元素上。

```

Loop: L. D      F0, 0(R1)    // 取一个数组元素放入 F0
      ADD. D    F4, F0, F2    // 加上在 F2 中的标量
      S. D      F4, 0(R1)    // 存结果
      DADDIU    R1, R1, #-8    // 将指针减少 8(每个数据占 8B)
      BNE       R1, R2, Loop  // 若 R1 不等于 R2, 表示尚未结束, 转移
                                // 到 Loop 继续。

```

现做以下假设:

- ① 每个时钟周期能流出一条整数指令和一条浮点指令, 即使它们相关也是如此。
- ② 有一个整数部件, 用于整数 ALU 运算和地址计算, 并且对于每一种浮点操作类型都有一个独立的流水化了的浮点功能部件。
- ③ 指令流出和写结果各占用一个时钟周期。
- ④ 具有动态分支预测部件和一个独立的计算分支条件的功能部件。
- ⑤ 分支指令单独流出, 没有采用延迟分支, 但分支预测是完美的。分支指令完成前, 其后续指令只能被取出和流出, 但不能执行。
- ⑥ 因为写结果占用一个时钟周期, 所以产生结果的延迟为整数运算一个周期, load 两个周期, 浮点加法运算 3 个周期。

列出该程序前面 3 遍循环中各条指令的流出、开始执行和将结果写到 CDB 上的时间。

解 执行时, 该循环将动态展开, 并且只要可能就流出两条指令。为了便于分析, 列出了访存发生的时刻。运行结果如图 4.13 所示。

说明:

- ① store 和分支指令不经过写结果段, 因为它们不写寄存器。
 - ② 对于 load 和 store 来说, 在执行段是进行有效地址计算。
 - ③ 对于分支指令来说, 在执行段是进行转移条件判定, 并检测分支预测是否正确。假设它所用的操作数一旦就绪, 就可以立即执行。
 - ④ 需注意, 下一个循环迭代中的 load 指令比当前迭代的 store 指令先访问存储器。
- 从图 4.13 中可以看出, 程序基本可以达到 3 拍流出 5 条指令, $IPC = 5/3 = 1.67$ 条/拍。虽然指令的流出率比较高, 但是执行效率并不是很高, 16 拍共执行 15 条指令, 平均指令执行速度为 $15/16 = 0.94$ 条/拍。

这相对于单流出流水线来说改进不大, 其原因是浮点运算少, ALU 部件成了瓶颈。

为解决这个问题, 可以考虑在此结构的基础上增加一个加法器, 把 ALU 功能和地址运算功能分开。不过, 这样做会出现多条指令同时写 CDB 的情况, 也就是出现了 CDB 竞争。在增加了一条 CDB 以后竞争消失, 这时会发现已经在原有结构的基础上增加了大量的硬件, 系统开始变得很复杂了。与此同时, 各个部件的使用效率问题就会列

人考虑的范围。

遍数	指 令	流出	执行	访存	写 CDB	说明
1	L. D F0,0(R1)	1	2	3	4	流出第一条指令
1	ADD. D F4,F0,F2	1	5		8	等待 L. D 的结果
1	S. D F4,0(R1)	2	3	9		等待 ADD. D 的结果
1	DADDIU R1,R1,#-8	2	4		5	等待 ALU(计算指令 S. D 的有效地址也是用该 ALU)
1	BNE R1,R2,Loop	3	6			等待 DADDIU 的结果
2	L. D F0,0(R1)	4	7	8	9	等待 BNE 完成
2	ADD. D F4,F0,F2	4	10		13	等待 L. D 的结果
2	S. D F4,0(R1)	5	8	14		等待 ADD. D 的结果
2	DADDIU R1,R1,#-8	5	9		10	等待 ALU
2	BNE R1,R2,Loop	6	11			等待 DADDIU 的结果
3	L. D F0,0(R1)	7	12	13	14	等待 BNE 完成
3	ADD. D F4,F0,F2	7	15		18	等待 L. D 的结果
3	S. D F4,0(R1)	8	13	19		等待 ADD. D 的结果
3	DADDIU R1,R1,#-8	8	14		15	等待 ALU
3	BNE R1,R2,Loop	9	16			等待 DADDIU 的结果

图 4.13 基于 Tomasulo 算法的双流出超标量流水线中指令的流出、执行和写 CDB 时钟分析

上述双流出动态调度流水线的性能受限于以下 3 个因素：

- ① 整数部件和浮点部件的工作负载不平衡,没有充分发挥出浮点部件的作用。应该设法减少循环中整数型指令的数量。
- ② 每个循环迭代中的控制开销太大。5 条指令中有两条指令是辅助指令。应该设法减少或消除这些指令。4.5 节中将介绍减少这种开销的方法。
- ③ 控制相关使得处理机必须等到分支指令的结果出来后才能开始下一条 LD 指令的执行。采用 4.3.3 小节中的前瞻执行可以解决这个问题。

4.4.3 超长指令字技术

超长指令字(Very Long Instruction Word, VLIW)技术是另一种多指令流出技术。与超标量不同,它在指令流出时不需要进行复杂的冲突检测,而是依靠编译器在编译时找出指令之间潜在的并行性,并通过指令调度把可能出现的数据冲突减少到最少,最后

把能并行执行的多条指令组装成一条很长的指令。这种指令字经常是 100 多位到几百位,超长指令字因此得名。

在 VLIW 处理机中一般设置有多个功能部件。相应地,指令字也被分割成一些字段,每个字段称为一个操作槽,直接独立地控制一个功能部件。为了使功能部件充分忙碌,程序指令序列中应有足够的并行性,从而尽量填满每个操作槽。这种并行性是依靠编译器来挖掘的。编译器把循环展开,并在展开后的代码中进行指令调度。

在 VLIW 处理机中,所有的处理和指令安排都是由编译器完成的,它不需要超标量处理机中用于指令流出控制的硬件,因而控制硬件比较简单。特别是当流出宽度增加时,VLIW 技术的优点更加明显。

例 4.5 假设 VLIW 处理机每个时钟周期可同时流出 5 条指令:两条访存指令、两条浮点操作指令和一条整数指令或分支指令。对于例 4.4 中的循环展开后的代码,给出它在该 VLIW 中的代码序列。不考虑分支指令的延迟槽。

解 代码序列如图 4.14 所示。

访存指令 1	访存指令 2	浮点指令 1	浮点指令 2	整数/转移指令
L.D F0,0(R1)	L.D F6,-8(R1)			
L.D F10,-16(R1)	L.D F14,-24(R1)			
L.D F18,-32(R1)		ADD.D F4,F0,F2	ADD.D F8,F6,F2	
		ADD.D F12,F10,F2	ADD.D F16,F14,F2	
		ADD.D F20,F18,F2		
S.D F4,0(R1)	S.D F8,-8(R1)			
S.D F12,-16(R1)	S.D F16,-24(R1)			DADDIU R1,R1,#-40
S.D F20,8(R1)				BNE R1,R2,Loop

图 4.14 循环展开后的 VLIW 代码序列

这段程序的运行时间为 8 个时钟周期,每遍循环平均 1.6 个时钟周期。8 个时钟周期内流出了 17 条指令,每个时钟周期 2.1 条。8 个时钟周期共有操作槽 $8 \times 5 = 40$ 个,有效槽的比例为 42.5%。

当然,VLIW 也存在以下 3 个问题。

① 程序代码长度增加。有两个因素导致代码长度的增加,一个是为了提高并行性而进行的大量的循环展开,另一个是指令字中的操作槽并非总能填满。虽然编译器会尽量使各操作部件充分忙碌,但总会出现无法安排指令到操作部件执行的情况,这时指令字中与该操作部件对应的操作槽就没有被利用起来,这会造成空间上的浪费。

为了解决这个问题,可采用指令共享立即数字段的方法,或者采用指令压缩存储、调入 Cache 或译码时展开的方法。

② 采用了锁步机制。由于没有冲突检测硬件,早期的 VLIW 都采用了锁步机制(lockstep)。即任何一个操作部件出现停顿时,整个处理机都要停顿,这是因为所有的功能部件是同步操作的。当指令流出速率和访存数量增加很多时,这个同步限制条件就变得让人无法接受了。在新近的 VLIW 处理机中,各功能部件有更多的独立性。可以通过设置适当的硬件动态检测机制来允许指令流出后的非同步执行。

③ 机器代码的不兼容性。在 VLIW 中,编译生成代码时是充分利用了指令集的特点以及具体流水线结构的细节(如指令流出数目、功能单元延迟时间等都不同)来进行优化的,所以在不同配置的 VLIW 计算机中,机器代码差别很大。即使是同一系列的 VLIW 计算机也难以实现机器代码的兼容。而超标量处理机在这方面则好很多。

为了解决这个问题,可以采用机器代码翻译或者仿真的方法。这两种方法实际上是解决所有移植问题的通用方法。

4.4.4 多流出处理器受到的限制

指令多流出处理器受哪些因素的限制呢?既然每个时钟周期可以流出 5 条指令,为什么不能流出 50 条呢?实际上,处理器中指令的流出能力是有限的,它主要受以下 3 个方面的影响:

- 程序所固有的指令级并行性。
- 硬件实现上的困难。
- 超标量和超长指令字处理器固有的技术限制。

其中,第一个限制是最简单的,也是最根本的因素。对于流水线处理器,需要有大量可并行执行的操作才能避免流水线出现停顿。如果浮点流水线的延迟为 5 个时钟周期,要使该浮点流水线不停顿,就必须有 5 条无相关的浮点指令。通常情况下,所需要的无相关指令数等于流水线的深度乘以可以同时工作的功能部件数。例如,要使具有 5 个功能部件的流水线充分忙碌,需要连续 15~25 条无相关指令。

第二个限制是指多指令流出的处理器需要大量的硬件资源。因为每个时钟周期不仅要流出多条指令,而且还要执行它们。随着每个时钟周期流出指令数的增加,所需的硬件成正比例地增长,这包括控制流出的硬件、整数运算部件和浮点运算部件等。同时,所需的存储器带宽和寄存器带宽也大大增加了,这样的带宽要求必然导致大量增加硅片面积,加大面积将导致产生时钟频率下降、功耗增加、可靠性降低等一系列问题。在例 4.5 中的 5 个功能单元的 VLIW 处理机中,由于有两个访存指令同时执行,就需要两个存储器读端口,它比寄存器端口的硬件成本大得多。如果要使流出指令的数目增加,就需要进一步增加更多的存储器端口。这时,只增加运算部件是没用的,因为这时处理器受限于存储器的带宽。多端口、层次化的存储系统带来的系统复杂性和访问延迟,可能是超标量和超长指令字处理器等指令多流出技术所面临的最严重的硬件限制。

多指令流出所需的硬件量随实现方法的不同有很大的差别。一个极端是动态调度的超标量处理器,无论采用记分牌技术还是 Tomasulo 算法,都需要大量的硬件,而且动态调度也大大增加了设计的复杂性,使提高时钟频率更加困难。另一个极端是 VLIW 处理器,指令的流出和调度仅需要很少甚至不需要额外的硬件,因为这些工作全都由编译器进行。这两种极端之间是现存的多数超标量处理器,它们将编译器的静态调度和硬件的动态调度机制结合起来,共同决定可同时并行流出的指令数。设计多流出处理器的主要难点是访存的开销、硬件的复杂性和编译器技术的难度。各个因素的权衡和技术的取舍往往取决于设计人员认为它们对性能有多大影响。

最后是超标量处理器和 VLIW 所固有的限制。在第二个限制中已经讨论了超标量处理器所存在的问题。对于 VLIW,其存在的问题见 4.4.3 小节。

4.4.5 超流水线处理机

在第 3 章介绍的流水处理机中,把一条指令的执行过程分解为取指令、译码、执行、访存、写结果 5 个流水段。如果把其中的每个流水段进一步细分,例如,分解为两个延迟时间更短的流水段,则一条指令的执行过程就要经过 10 个流水段。这样,在一个时钟周期内,取指令、译码、执行、访存、写结果等各段都在处理各自的两条指令。这种在一个时钟周期内能够分时流出多条指令的处理机称为超流水线处理机(superpipelining)。

前面介绍的超标量处理机是通过重复设置多份硬件来提高性能的,即设置多个取指部件,设置多个译码、执行、访存、写结果等部件,并让它们同时工作。它需要增加比较多的硬件。而超流水线处理机则不同,它只需要增加少量硬件,是通过各部分硬件的充分重叠工作来提高性能的。超标量处理机采用的是空间并行性,而超流水线处理机采用的是时间并行性。

对于一台每个时钟周期能流出 n 条指令的超流水线计算机来说,这 n 条指令不是同时流出的,而是每隔 $1/n$ 个时钟周期流出一条指令。因此,实际上该超流水线计算机的流水线周期为 $1/n$ 个时钟周期。图 4.15 是一台每个时钟周期分时流出两条指令的超流水线计算机的时空图。该图只是超流水线计算机原理上的指令执行时空图。实际上,流水段还要进一步细分,将一个流水段再细分为多个流水级(本小节用流水“级”来表示流水“段”进一步细分后的结果),每一个流水级也都有名称。在分解流水段时,根据实际情况,各段的级数可以多少不一。例如将取指令段再细分为“取指 1”、“取指 2”两个级,把译码段再细分为“译码”、“取操作数”两个级等,而 WB 段一般不再细分。在有的资料中,把指令流水线级数为 8 或 8 以上的流水线处理机称为超流水线处理机。

在目前大量使用的微处理器中,SGI 公司的 MIPS 系列 R4000 是典型的超流水线处理器。R4000 微处理器芯片内有两个 Cache:指令 Cache 和数据 Cache,容量都是 8 KB,每个 Cache 的数据宽度为 64 位。

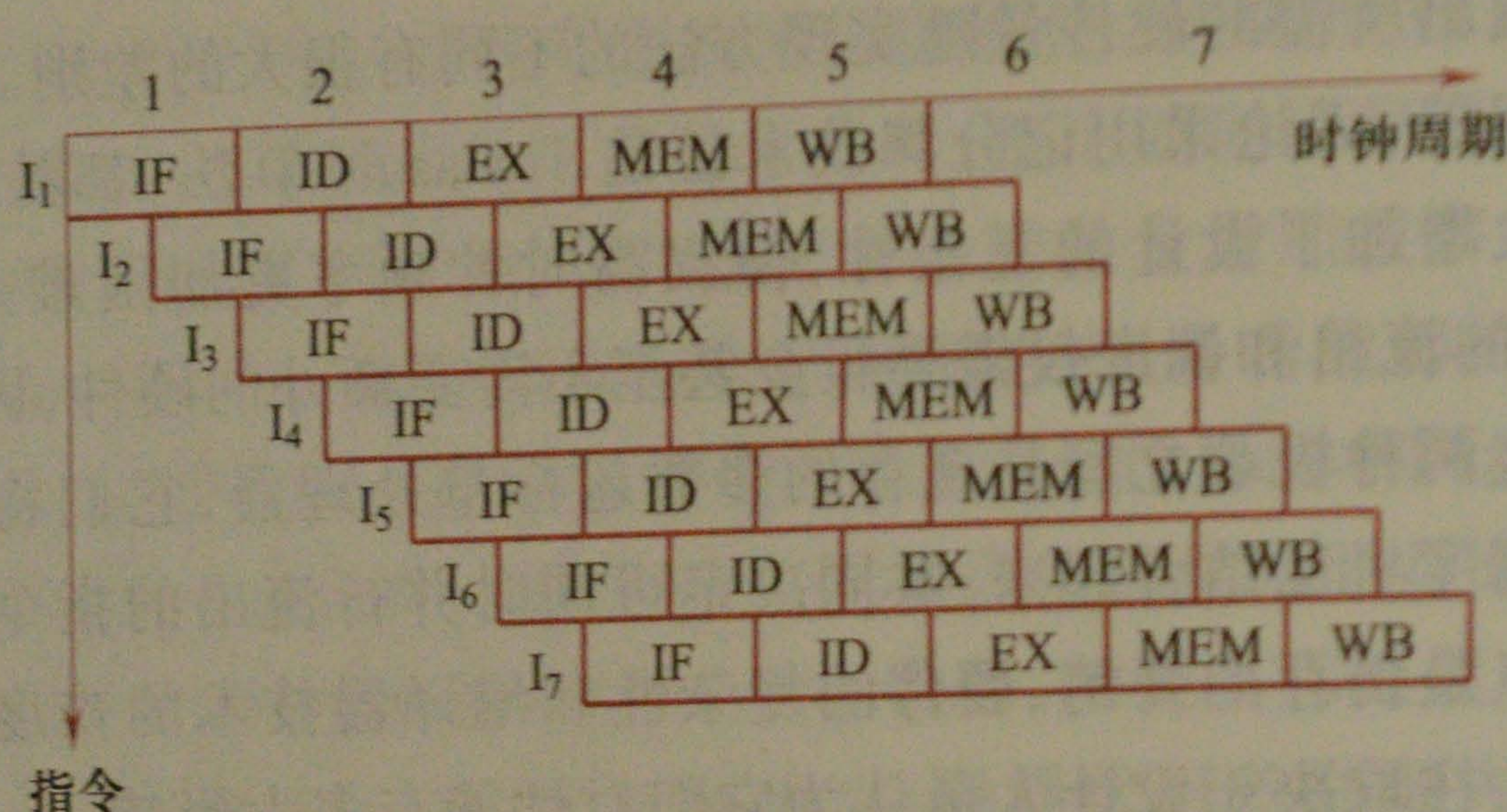


图 4.15 超流水线处理机的指令执行时空图

整数部件是 R4000 的核心处理部件,它主要包括一个 32×32 位的通用寄存器组、一个算术逻辑部件(ALU)、一个专用的乘法/除法部件。整数部件负责取指令、整数操作的译码和执行、load/store 操作的执行等。通用寄存器组有两个输出端口和一个输入端口,用于标量整数操作和地址计算,并设置有专用的数据通路,可以对每一个寄存器读/写两次。算术逻辑部件包括一个整数加法器和一个逻辑部件,负责执行算术运算操作、地址运算和所有的移位操作。乘法/除法部件能够执行 32 位带符号和不带符号的乘法或除法操作,它可以和 ALU 部件并行地执行指令。

浮点部件包括一个执行部件和一个 16×64 位的浮点通用寄存器组。浮点通用寄存器组也可以设置成 32 个 32 位的浮点寄存器。浮点执行部件由浮点乘法部件、浮点除法部件和浮点加法/转换/求平方根部件等 3 个独立的部件组成,它们可以并行工作。

R4000 的指令流水线有 8 级,如图 4.16 所示。这里把取指令段细分成 IF 和 IS 两级,把取数据也细分成 DF 和 DS 两级。各级的功能如下所示。

- IF:取指令的前半步,根据 PC 值去启动对指令 Cache 的访问。
- IS:取指令的后半步,在这一级完成对指令 Cache 的访问。
- RF:指令译码,访问寄存器组读取操作数,冲突检测,并判断指令 Cache 是否命中。
- EX:指令执行。包括有效地址计算、ALU 操作、分支目标地址计算和条件码测试。
- DF:取数据的前半步,启动对数据 Cache 的访问。

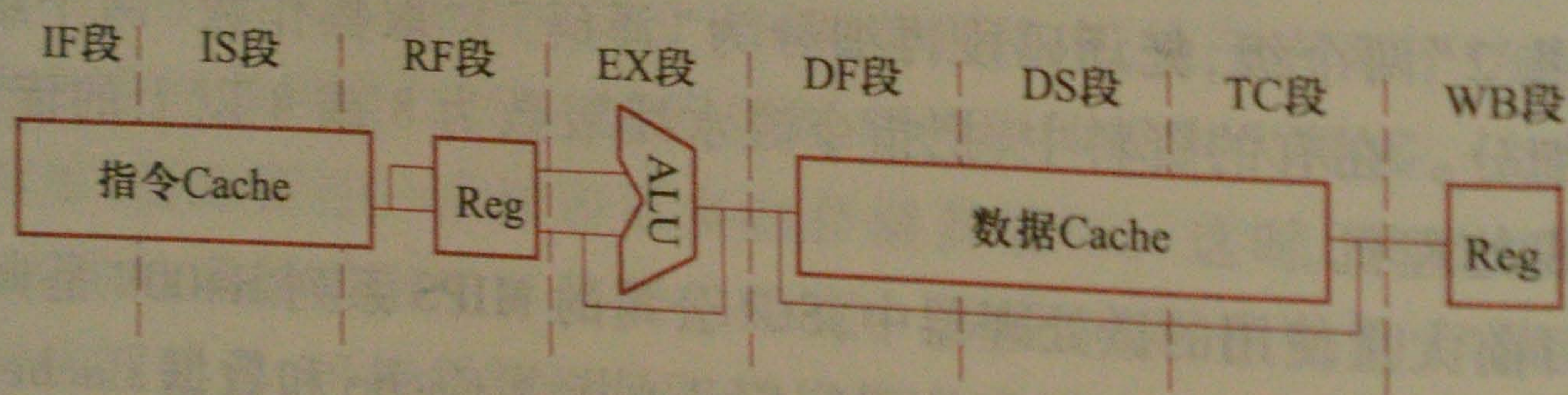


图 4.16 MIPS R4000 的 8 级流水线结构

- DS:取数据的后半步,在这一级完成对数据 Cache 的访问。
- TC:标识比较,判断对数据 Cache 的访问是否命中。
- WB:load 指令或运算型指令把结果写回寄存器组。

图 4.16 中垂直的虚线表示级与级之间的界限,也即流水线寄存器所在的位置。在该流水线中,指令在 IS 的末尾就可以使用了。不过判断指令 Cache 是否命中是在 RF 段执行的。对于非存储器访问指令,如果指令 Cache 命中,那么指令可以在 EX 流水级执行,执行结果可以在 EX 的末尾得到。对于存储器访问指令,则是在 DF 和 DS 流水级期间访问数据 Cache,这时存储管理部件(MMU)要在这两个流水级中把数据的虚拟地址变换成主存物理地址,然后在 TC 段把该物理地址与从数据 Cache 中读出的标识(tag)进行比较,判断是否命中(关于 Cache 的知识见第 5 章“存储系统”)。对于 store 指令来说,如果命中,只要把数据送到写入缓冲器,由写入缓冲器负责把数据写到数据 Cache 的指定单元中去。对于非存储器操作指令,在 WB(写结果)流水级把指令的最后执行结果写回到通用寄存器组中。

正常情况下,MIPS R4000 指令流水线时空图如图 4.17 所示。一条指令的执行过程经历 8 个流水线周期。从流水线的输入端看,每一个流水线周期启动一条指令。同样,从流水线输出端看,每一个流水线周期执行完一条指令。当流水线被填满时,如图 4.17 中黑框内所示,有 8 条指令在同时执行。如果把两个流水线周期看作是一个时钟周期,则在一个时钟周期内分时流出了两条指令。同样,在一个时钟周期内也是完成了两条指令。

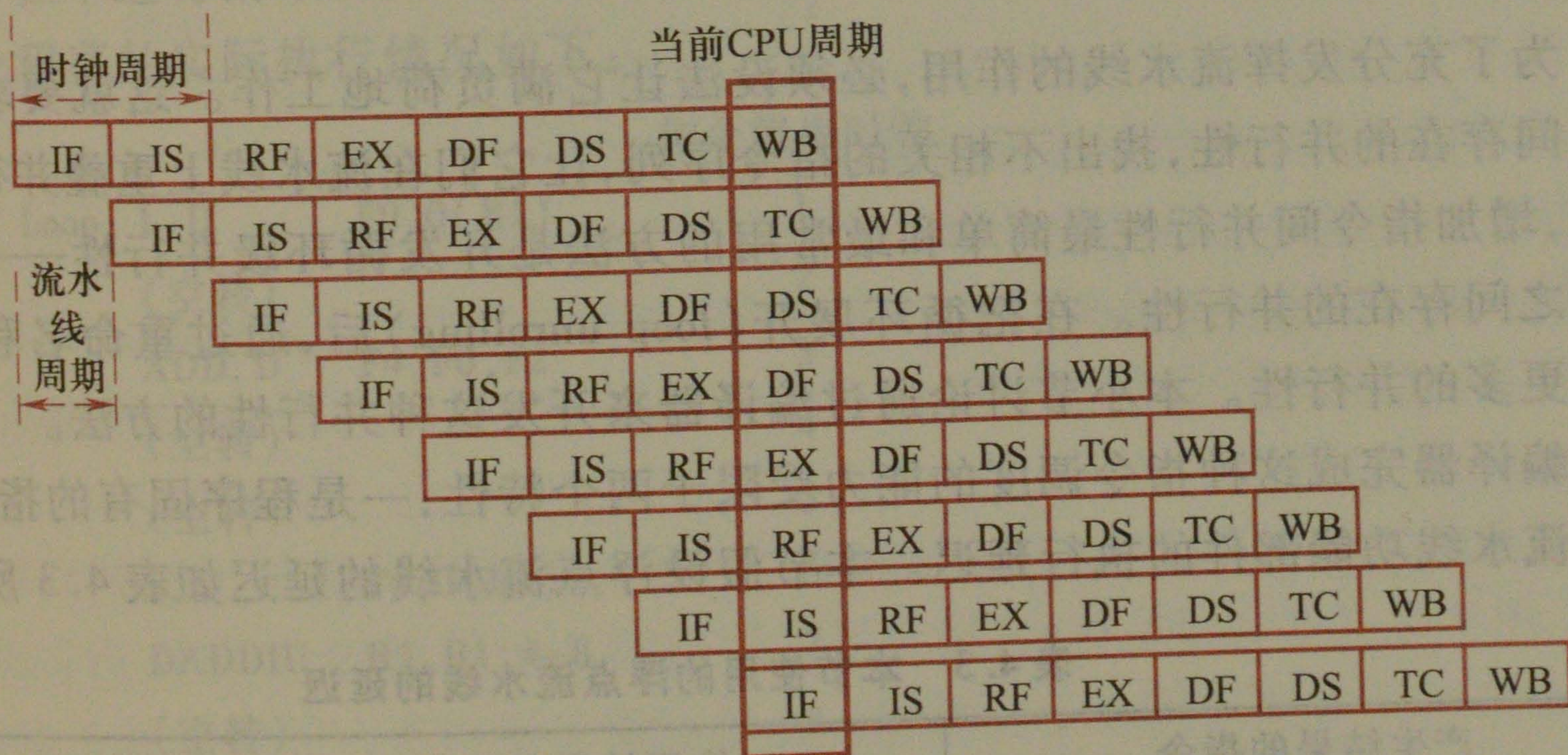


图 4.17 MIPS R4000 正常指令的流水线时空图

对于 load 指令,数据要在 DS 的末尾才能准备好。紧跟其后的指令若是需要使用该 load 取来的数据,就要等待两个流水线周期(已经采用了数据定向),如图 4.18 所示。可以看出,这时数据 Cache 是否命中的结果还没出来,流水线实际上超前使用了该数据。如果后面发现数据 Cache 不命中,流水线就要等正确的数据来到时回退一个时钟周期,然后继续执行。

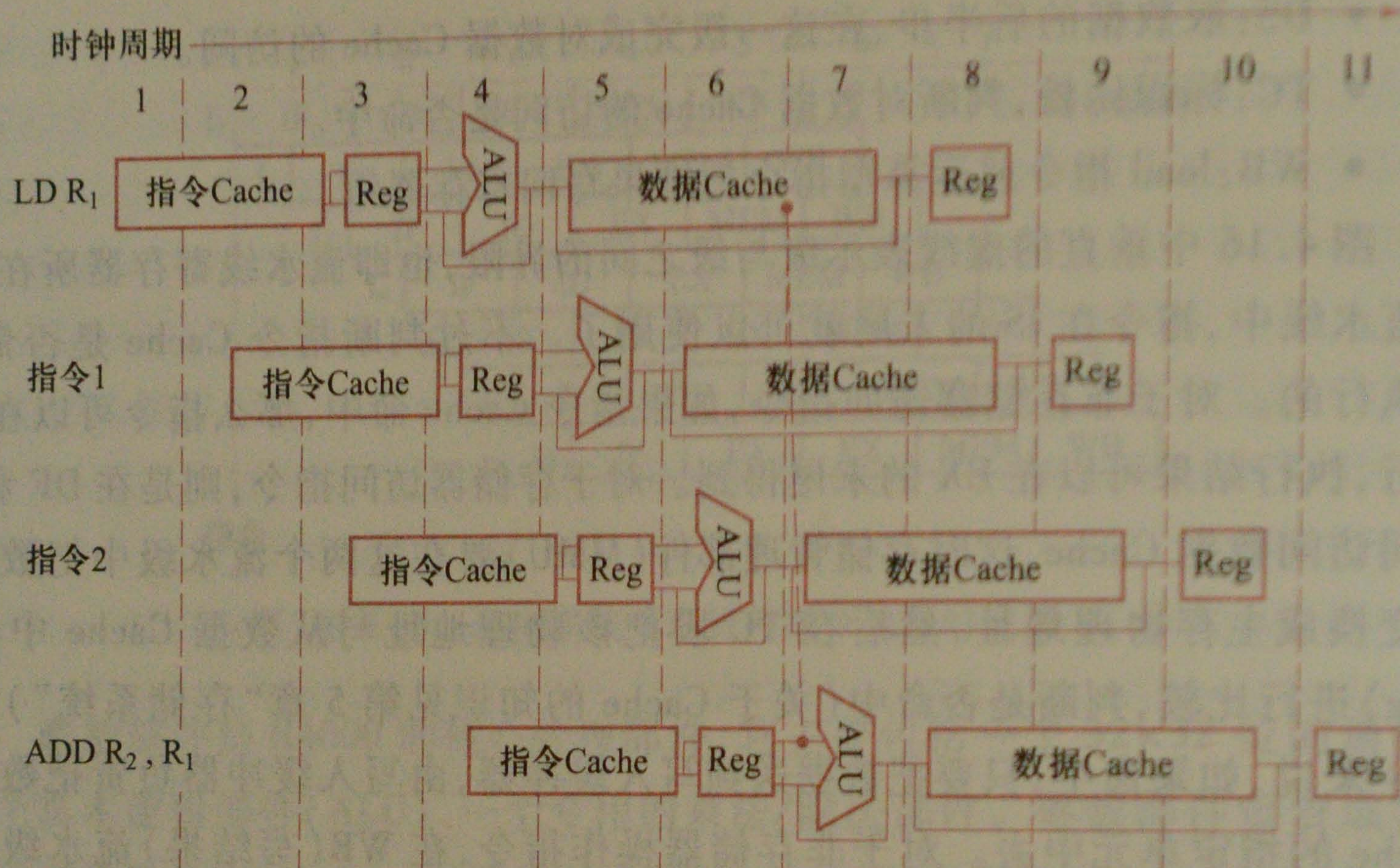


图 4.18 load 指令引起的流水线停顿

4.5 循环展开和指令调度

4.5.1 循环展开和指令调度的基本方法

为了充分发挥流水线的作用,必须设法让它满负荷地工作。这就要求充分开发指令之间存在的并行性,找出不相关的指令序列,让它们在流水线上重叠并行执行。前面讲过,增加指令间并行性最简单和最常用的方法是开发循环级并行性——循环的不同迭代之间存在的并行性。在把循环展开(loop unrolling)后,通过重命名和指令调度来开发更多的并行性。本小节讨论通过编译器来开发这种并行性的方法。

编译器完成这种指令调度的能力受限于两个特性:一是程序固有的指令级并行性;二是流水线功能部件的执行延迟。本节假设浮点流水线的延迟如表 4.3 所示。

表 4.3 本节使用的浮点流水线的延迟

产生结果的指令	使用结果的指令	延迟(时钟周期数)
浮点计算	另一个浮点计算	3
浮点计算	浮点 store(S, D)	2
浮点 load(L, D)	浮点计算	1
浮点 load(L, D)	浮点 store(S, D)	0

需要说明的是,由于浮点 load 指令的结果可以通过定向路径及时地送给浮点 store

指令,所以延迟为0(就是不用插入停顿)。跟以前一样,仍假设是采用图3.23的5段整数流水线,分支的延迟是一个时钟周期,整数load指令的延迟为一个时钟周期,并假设整数运算部件是全流水或者重复设置了足够的份数,因而每个时钟周期都能流出一条整数指令。

下面通过一个实例对循环展开技术进行研究和性能分析。

例4.6 对于下面的源代码,转换成MIPS汇编语言,在不进行指令调度和进行指令调度两种情况下,分析其代码一次循环所需的执行时间。

```
for (i=1; i<=1000; i++)
```

```
  x[i] = x[i] + s;
```

解 显然这一循环的不同迭代之间是不存在相关的,所以多次迭代可以并行执行。

先把该程序翻译成MIPS汇编语言代码,如下所示:

```
Loop: L.D      F0,0(R1)      // 取一个向量元素放入 F0
      ADD.D     F4,F0,F2      // 加上在 F2 中的标量
      S.D       F4,0(R1)      // 存结果
      DADDIU    R1,R1,#-8     // 将指针减 8(每个数据占8B)
      BNE       R1,R2,Loop    // 若 R1 不等于 R2,表示尚未结束,转移
                               // 到 Loop 继续。
```

其中,整数寄存器R1用于指向向量中的当前元素,其初值指向第一个元素,8(R2)指向最后一个元素,浮点寄存器F2用于保存常数s。

在不进行指令调度的情况下,根据表4.3中给出的浮点流水线中指令执行的延迟,以上程序的实际执行情况如下:

指令流出时钟

Loop: L.D	F0,0(R1)	1
(空转)		2
ADD.D	F4,F0,F2	3
(空转)		4
(空转)		5
S.D	F4,0(R1)	6
DADDIU	R1,R1,#-8	7
(空转)		8
BNE	R1,R2,Loop	9
(空转)		10

可以看出,每完成一个元素的操作需要10个时钟周期,其中有5个是空转周期。

在用编译器对上述程序进行指令调度以后,程序的执行情况如下:

指令流出时钟

Loop: L.D	F0,0(R1)	1
DADDIU	R1,R1,#-8	2

ADD. D	F4, F0, F2	3
(空转)		4
BNE	R1, R2, Loop	5
S. D	F4, 8(R1)	6

这里,把 DADDIU 指令调度到了 L. D 指令和 ADD. D 指令之间的“空转”拍。把 S. D 指令放到了分支指令的延迟槽中。由于修改指针的 DADDIU 指令被调度到 S. D 指令之前,提前对指针进行了减 8 的操作,所以要把 S. D 指令中的偏移量进行修正,即把“0(R1)”改为“8(R1)”。

经过这样的指令调度后,一个元素的操作时间从 10 个时钟周期减少到了 6 个时钟周期,其中 5 个周期是有指令执行的,只剩下一个空转周期。

进一步分析上面的例子可以发现,虽然完成对一个元素的操作时间从 10 个时钟周期减少到 6 个时钟周期,但是其中只有 L. D、ADD. D 和 S. D 这 3 条指令是需要的有效操作,占用 3 个时钟周期,而 DADDIU、空转和 BNE 这 3 个时钟周期都是为了控制循环和解决数据相关等待而附加的,因此整个执行过程中有效操作的比率并不高。

要减少这种附加的循环控制开销在程序执行时间中所占的比率,一个简单办法就是运用循环展开技术。所谓循环展开是指把循环体的代码复制多次并按顺序排放,然后相应调整循环的结束条件。

这种技术给编译器进行指令调度带来了更大的空间,多次循环的代码可以合到一起调度。而且消除了中间的分支指令。通过下面的例子可以更清楚地看到循环展开技术所带来的好处。

例 4.7 将上述例子中的循环展开 3 次得到 4 个循环体,然后对展开后的指令序列在不调度和调度两种情况下分析代码的性能。假定 R1 的初值为 32 的倍数,即循环次数为 4 的倍数。消除冗余的指令,并且不要重复使用寄存器。

解 由于假定 R1 的初值为 32 的倍数,即循环次数为 4 的倍数,因此在展开成 4 个循环体后,循环结束后没有剩余尚未执行的操作,因此无须在循环体后面增加补偿代码。

首先分配寄存器(注意不重复使用寄存器):F0、F4 已经用于展开后的第 1 个循环体,F2 用于保存常数,F6 和 F8 用于展开后的第 2 个循环体,F10 和 F12 用于第 3 个循环体,F14 和 F16 用于第 4 个循环体。这里不重复使用寄存器实际就是进行了重命名。展开后没有调度的代码如下。

		指令流出时钟
Loop: L. D	F0, 0(R1)	1
(空转)		2
ADD. D	F4, F0, F2	3
(空转)		4
(空转)		5

S. D	F4,0(R1)	6
L. D	F6,-8(R1)	7
(空转)		8
ADD. D	F8,F6,F2	9
(空转)		10
(空转)		11
S. D	F8,-8(R1)	12
L. D	F10,-16(R1)	13
(空转)		14
ADD. D	F12,F10,F2	15
(空转)		16
(空转)		17
S. D	F12,-16(R1)	18
L. D	F14,-24(R1)	19
(空转)		20
ADD. D	F16,F14,F2	21
(空转)		22
(空转)		23
S. D	F16,-24(R1)	24
DADDIU	R1,R1,#-32	25
(空转)		26
BNE	R1,R2,Loop	27
(空转)		28

这里把展开后的前 3 个循环体中的 DADDIU 指令删除了,并对 L. D 指令中的偏移量进行了相应修正。对最后留下的 DADDIU 指令中的立即数也进行了相应的调整。

这个循环每遍共使用了 28 个时钟周期,有 4 个循环体,完成 4 个元素的操作,平均每个元素使用 $28/4=7$ 个时钟周期。与原代码的每个元素需要 10 个时钟周期相比较,节省了不少的时间。这主要是从减少循环控制的开销中获得的。但是在展开后的循环体中,实际指令只有 14 条,其他 13 个周期都是空转,可见效率并不高。下面对指令序列进行优化调度,以减少空转周期。

指令流出时钟

Loop:L. D	F0,0(R1)	1
L. D	F6,-8(R1)	2
L. D	F10,-16(R1)	3
L. D	F14,-24(R1)	4
ADD. D	F4,F0,F2	5
ADD. D	F8,F6,F2	6
ADD. D	F12,F10,F2	7

ADD. D	F16, F14, F2	8
S. D	F4, 0(R1)	9
S. D	F8, -8(R1)	10
DADDIU	R1, R1, #-32	12
S. D	F12, 16(R1)	11
BNE	R1, R2, Loop	13
S. D	F16, 8(R1)	14

这个循环由于没有数据相关引起的空转等待,整个循环仅仅使用了 14 个时钟周期,平均每个元素使用 $14/4=3.5$ 个时钟周期。

从上述例子中可以知道,通过循环展开、寄存器重命名和指令调度,可以有效地开发出指令级并行。

循环展开和指令调度时要注意以下几个方面:

① 保证正确性。在循环展开和调度过程中尤其要注意两个地方的正确性:循环控制,操作数偏移量的修改。

② 注意有效性。只有能够找到不同循环体之间的无关性,才能够有效地使用循环展开。

③ 使用不同的寄存器。如果使用相同的寄存器,或者使用较少数量的寄存器,就可能产生新的冲突。

④ 删除多余的测试指令和分支指令,并对循环结束代码和新的循环体代码进行相应的修正。

⑤ 注意对存储器数据的相关性分析。例如,对于 load 指令和 store 指令,如果它们在不同的循环迭代中访问的存储器地址是不同的,它们就是相互独立的,可以相互对调。

⑥ 注意新的相关性。由于原循环不同次的迭代在展开后都到了同一次循环体中,因此可能带来新的相关性。

4.5.2 静态超标量处理机中的循环展开

下面来看看在表 4.3 中列出的延迟的条件下,超标量处理器如何进行循环展开和指令调度。

例 4.8 下面是前面使用的循环程序段,对其进行循环展开,并在超标量流水线上进行调度。

Loop: L. D	F0, 0(R1)	// 取一个数组元素放入 F0
ADD. D	F4, F0, F2	// 加上在 F2 中的标量
S. D	F4, 0(R1)	// 存结果
DADDIU	R1, R1, #-8	// 将指针减少 8(每个数据占 8B)
BNE	R1, R2, Loop	// 若 R1 不等于 R2,表示尚未结束,转移

// 到 Loop 继续。

解 要达到无延迟的指令调度,须将 5 遍循环展开。这样才能使 S. D 指令(使用 ADD. D 的结果)不会停顿。采用类似于例 4.7 的方法进行展开和调度,可得如下代码:

```

Loop: L. D      F0, 0(R1)
      L. D      F6, -8(R1)
      L. D      F10, -16(R1)
      L. D      F14, -24(R1)
      L. D      F18, -32(R1)
      ADD. D     F4, F0, F2
      ADD. D     F8, F6, F2
      ADD. D     F12, F10, F2
      ADD. D     F16, F14, F2
      ADD. D     F20, F18, F2
      S. D       F4, 0(R1)
      S. D       F8, -8(R1)
      S. D       F12, -16(R1)
      S. D       F16, -24(R1)
      DADDIU     R1, R1, #-40
      BNE        R1, R2, Loop
      S. D       F20, 8(R1)
  
```

进一步针对超标量进行调度后的指令序列如图 4.19 所示。

	整数指令	浮点指令	时钟周期
Loop:	L. D F0, 0(R1)		1
	L. D F6, -8(R1)		2
	L. D F10, -16(R1)	ADD. D F4, F0, F2	3
	L. D F14, -24(R1)	ADD. D F8, F6, F2	4
	L. D F18, -32(R1)	ADD. D F12, F10, F2	5
	S. D F4, 0(R1)	ADD. D F16, F14, F2	6
	S. D F8, -8(R1)	ADD. D F20, F18, F2	7
	S. D F12, -16(R1)		8
	S. D F16, -24(R1)		9
	DADDIU R1, R1, #-40		10
	BNE R1, R2, Loop		11
	S. D F20, 8(R1)		12

图 4.19 在超标量流水线上展开并调度后的代码

在超标量流水线上展开的代码每次循环需 12 个时钟周期,即计算每个结果需要 2.4 个时钟周期(每次循环计算 5 个结果)。而在普通的 MIPS 流水线上,没有调度的代码迭代一次为 9 个时钟周期,调度后为 6 个时钟周期,展开 4 次并调度后每个迭代为 3.5 个时钟周期。与之相比,超标量流水线的性能提高分别为 3.75 倍、2.5 倍和 1.4 倍。

在这个例子中可以看到,超标量 MIPS 流水线的性能主要受限于整数计算和浮点计算之间的平衡问题。本例中没有足够的浮点指令来使两路流水线都达到饱和。

习题 4

4.1 解释下列术语

指令级并行	指令调度	指令的动态调度	指令的静态调度
保留站	CDB	动态分支预测技术	BHT
分支目标缓冲	前瞻执行	ROB	超标量
超流水	超长指令字	循环展开	

4.2 简述 Tomasulo 算法的基本思想。

4.3 根据需要展开下面的循环并进行指令调度,直到没有任何延迟。指令的延迟如表 4.3 所示。

```

LOOP:  L.D      F0,0(R1)
        MUL.D   F0,F0,F2
        L.D      F4,0(R2)
        ADD.D   F0,F0,F4
        S.D      F0,0(R2)
        DSUBI   R1,R1,#8
        DSUBI   R2,R2,#8
        BNEZ    R1,LOOP
  
```

4.4 假设有一条长流水线,仅仅对条件转移指令使用分支目标缓冲。假设分支预测错误的开销为 4 个时钟周期,缓冲不命中的开销为 3 个时钟周期。假设命中率为 90%,预测精度为 90%,分支频率为 15%,没有分支的基本 CPI 为 1。

(1) 求程序执行的 CPI。

(2) 相对于采用固定的两个时钟周期延迟的分支处理,哪种方法程序执行速度更快?

4.5 假设分支目标缓冲的命中率为 90%,程序中无条件转移指令的比例为 5%,没有无条件转移指令的程序 CPI 值为 1。假设分支目标缓冲中包含分支目标指令,允许无条件转移指令进入分支目标缓冲,则程序的 CPI 值为多少?

4.6 下面的一段 MIPS 汇编程序是计算高斯消去法中的关键一步,用于完成下面公式的计算:

$$Y = a \times X + Y$$

其浮点指令延迟如表 4.3 所示,整数指令均为 1 个时钟周期完成,浮点和整数部件均采用流水。整数操作之间以及与其他所有浮点操作之间的延迟为 0,转移指令的延迟为 0。X 中的最后一个元素存放在存储器中的地址为 DONE。

```

FOO:  L. D      F2,0(R1)
      MUL. D    F4,F2,F0
      L. D      F6,0(R2)
      ADD. D    F6,F4,F6
      S. D      F6,0[R2]
      DADDIU    R1,R1,#8
      DADDIU    R2,R2,#8
      DSUBIU    R3,R1,#DONE
      BNEZ      R3,FOO
  
```

(1) 对于标准的 MIPS 单流水线,上述循环计算一个 Y 值需要多少时间? 其中有多少空转周期?

(2) 对于标准的 MIPS 单流水线,将上述循环顺序展开 4 次,不进行任何指令调度,计算一个 Y 值平均需要多少时间? 加速比是多少? 其加速是如何获得的?

(3) 对于标准的 MIPS 单流水线,将上述循环顺序展开 4 次,优化和调度指令,使循环处理时间达到最优,计算一个 Y 值平均需要多少时间? 加速比是多少?

(4) 对于采用图 4.9 的前瞻执行机制的 MIPS 处理器(只有一个整数部件),当第二次循环执行到

```
BNEZ R3,FOO
```

时,写出前面所有指令的状态,包括指令使用的保留站、指令起始节拍、执行节拍和写结果节拍,并写出处理器当前的状态。

(5) 对于两路超标量的 MIPS 流水线,设有两个指令流出部件,可以流出任意组合的指令,系统中的功能部件数量不受限制。将上述循环展开 4 次,优化和调度指令,使循环处理时间达到最优。计算一个 Y 值平均需要多少时间? 加速比是多少?

(6) 对于如图 4.13 结构的超长指令字 MIPS 处理器,将上述循环展开 4 次,优化和调度指令,使循环处理时间达到最优。计算一个 Y 值平均需要多少时间? 加速比是多少?

4.7 对于两路超标量处理器,从存储器取数据有两拍附加延迟,其他操作均有 1 拍附加延迟,对于下列代码,请按要求进行指令调度。

```

LW      R4,(R5)
LW      R7,(R8)
DADD    R9,R4,R7
LD      R10,(R11)
DMUL    R12,R13,R14
DSUB    R2,R3,R1
SW      R15,(R2)
DMUL    R21,R4,R7
SW      R23,(R22)
  
```


SW R21, (R24)

(1) 假设两路功能部件中同时最多只有一路可以是访问存储器的操作,同时也最多只有一路可以是运算操作,指令顺序不变。

(2) 假设两路功能部件均可以执行任何操作,指令顺序不变。

(3) 假设指令窗口足够大,指令可以乱序(out-of-order)流出,两路功能部件均可以执行任何操作。

4.8 对于例 4.5,在相同的条件下,如果展开 7 遍循环,求:

(1) 每遍循环的平均时钟周期。

(2) 每个时钟周期流出指令数。

(3) 操作槽(功能部件)的使用效率。

(4) 如果展开 10 遍,会出现哪些问题?

4.9 设指令流水线由取指令、分析指令和执行指令 3 个部件构成,每个部件经过的时间为 Δt ,连续流入 12 条指令。分别画出标量流水处理机以及 ILP 均为 4 的超标量处理机、超长指令字处理机、超流水处理机的时空图,并分别计算它们相对于标量流水处理机的加速比。